

Integration test

1. Unit Test (smallest level)

Definition:

Tests a *single piece of code in isolation* — usually a function or method.

Key idea:

“Does this function work correctly on its own?”

Characteristics:

- Tests **only one unit**
- **No real DB / API / Redis**
- Uses **mocks/stubs**
- Very fast
- Easy to debug

Unit testing verifies individual components in isolation using mocks, while integration testing verifies how multiple components work together, often involving real dependencies like databases or APIs.

2. Integration Test (combining parts)

Definition:

Tests how multiple components work **together**

Key idea:

“Do these modules work correctly when connected?”

Example (your backend case):

Instead of testing just a function, you test:

Route → Controller → Service → DB

POST /book-ticket

Integration test:

```
request(app)
  .post("/book-ticket")
  .send({ userId, eventId })
  .expect(200);
```

Now this test checks:

- Route works
- Controller logic works
- DB interaction works
- Full flow works

Most beginners:

Only write unit tests → think system is working

Reality:

System fails when components interact That's

why:

- Unit test = correctness
- Integration test = reality check

Downsides of integration tests (short):

- **Slow** – involve DB, APIs, full app flow
 - **Hard to debug** – failure source unclear
 - **Flaky** – can fail due to environment/timing issues
 - **Complex setup** – need DB, configs, test data
 - **Dependency issues** – external services may fail
 - **Data management** – must clean/reset state properly
- Bootstrap
Integration Test from scratch for Express Ts Prisma**

Step 1: Create folder and initialize

```
project mkdir express-prisma-vitest cd  
express-prisma-vitest npm init -y
```

This creates a new folder and a package.json with default settings.

Step 2: Initialize TypeScript

```
npx tsc --init
```

This creates tsconfig.json. Now, let's **set rootDir and outDir** so compiled JS goes into dist and TS source is in src.

Edit tsconfig.json and add/update:

```
"rootDir": "./src",  
"outDir": "./dist",
```

Package.json

```
scripts:{  
  "test" : "vitest"  
}
```

Step 3: Install runtime dependencies npm

```
install express @prisma/client pg dotenv  
@prisma/adapters-pg
```

- express → web framework
- @prisma/client → Prisma ORM client
- pg → PostgreSQL driver
- dotenv → for environment variables

Step 4: Install dev dependencies

`npm install -D prisma typescript ts-node @types/node @types/express vitest supertest`

- `prisma` → Prisma CLI
- `typescript, ts-node` → TypeScript compiler and runtime
- `@types/node, @types/express` → TypeScript type definitions
- `vitest` → test runner
- `supertest` → integration testing for HTTP endpoints

Step 5: Initialize Prisma

`npx prisma init`

This will create:

- `prisma/schema.prisma` → your database schema
- `.env` → for your database URL

At this point, your **project structure** looks like this:

```
express-prisma-vitest/  
├─ node_modules/  
├─ prisma/  
|   └─ schema.prisma  
├─ src/  
├─ package.json  
├─ tsconfig.json  
└─ .env
```

Step 6 : Create a file `docker-compose.yml` in your project root:

`services:`

`db:`

`image: postgres:15`

environment:

POSTGRES_USER: test

POSTGRES_PASSWORD:

test

POSTGRES_DB: testdb

ports:

- "5432:5432"

Explanation:

- image: postgres:15 → pulls PostgreSQL v15
- POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB → simple credentials
- ports → maps container DB port 5432 to your local machine

Since we're using **Postgres in Docker** as per your docker-compose.yml (user: test, password: test, db: testdb), the **DB URL for Prisma** is:

`DATABASE_URL="postgresql://test:test@localhost:5432/testdb"`

In .env

Perfect! Let's write a **minimal Prisma schema** for your app.

Edit prisma/schema.prisma:

```
model User {  
  id Int @id @default(autoincrement())  
  name String  
  email String @unique  
}
```

In src/prisma/client.ts

```
import "dotenv/config";  
import { PrismaPg } from "@prisma/adapters-pg";
```

```
import { PrismaClient } from "../generated/prisma/client";
const adapter = new PrismaPg({ connectionString: process.env.DATABASE_URL
});
const prisma = new PrismaClient({ adapter });
export { prisma };
```

Step 7: Build App Logic

Files created:

src/app.ts

```
import express from "express";
import {prisma} from
"./prisma/client"
import dotenv from "dotenv";
dotenv.config();

const app = express();
app.use(express.json());
// Route to get all users
app.get("/users", async (req,
res) => {
  const users = await
prisma.user.findMany();
  res.json(users);
});
// Route to create a user
app.post("/users", async (req,
res) => {
```

```
const { name, email } =
req.body;

try {

const user = await
prisma.user.create({ data: {
name, email } });

res.status(201).json(user);

} catch (err: any) {

res.status(400).json({ error:
err.message });

}

});

export default app;

src/server.ts

import app from "../app";

const PORT = process.env.PORT || 3000;

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

Step 8: Write Integration Tests

Create a folder tests/ and inside it, user.test.ts:

```
// tests/user.test.ts
import
request from "supertest";
import { describe, it, beforeEach, expect } from "vitest";
import app from "../app";
import { prisma } from "../prisma/client"
```

```

describe("User API", () => {

  // Clean DB before each test
  beforeEach(async () => {
    await prisma.user.deleteMany();
  });

  it("should create a new user", async () => {
    const res = await request(app)
      .post("/users")
      .send({ name: "Alice", email: "alice@test.com" });

    expect(res.status).toBe(201);
    expect(res.body).toHaveProperty("id");
    expect(res.body.name).toBe("Alice");
  });

  it("should list all users", async () => {
    // Create a user first
    await prisma.user.create({ data: { name: "Bob", email: "bob@test.com" } });

    const res = await request(app).get("/users");

    expect(res.status).toBe(200);
    expect(res.body.length).toBe(1);
    expect(res.body[0].name).toBe("Bob");
  });
});

```

will:

- Start Postgres
- Set up the DB
- Run migrations

- Execute tests
- Tear down Docker all in one command

Step : 8 Create: run-all.js

☒ Step 2 — Paste this code

```
const { execSync } = require("child_process");

const PROJECT = "test-env";

function run(cmd) {
  console.log(`\n🚀 ${cmd}`);
  execSync(cmd, { stdio: "inherit" });
}

(async () => {
  try {
    console.log("Starting PostgreSQL via Docker...");
    run(`docker compose -p ${PROJECT} up -d`);

    console.log("Waiting for DB...");
    await new Promise((res) => setTimeout(res, 10000));

    run("npx prisma generate");
    run("npx prisma migrate dev --name init");
    run("npm run test");
```

```
} catch (err) {  
  console.error(" ❌ Error occurred:", err.message);  
} finally {  
  console.log("Bringing down Docker containers...");  
  try {  
    run(`docker compose -p ${PROJECT} down`);  
  } catch (e) {  
    console.error("Failed to stop containers");  
  }  
  console.log("All done!");  
}  
})();
```

Run it

```
node run-all.js
```

⚠ Small but important

Make sure your package.json has:

```
"type": "module"
```

End to end testing with cypress

Step 1: Ensure App is Ready

Before we even touch Cypress, we need to make sure the **backend app is fully ready**. This step is critical for E2E testing because Cypress tests the **running server**, not just the code in memory.

1a: Project folder

Your project folder structure should look like this:

```
express-prisma-vitest/  
├─ node_modules/  
├─ prisma/  
|   └─ schema.prisma  
├─ src/  
|   ├─ app.ts  
|   └─ server.ts  
├─ tests/ (for Vitest)  
├─ package.json  
├─ tsconfig.json  
├─ .env  
└─ docker-compose.yml
```

1b: Check your Express app

src/app.ts should have:

- express.json() middleware
- prisma client initialized
- /users routes (GET and POST)
- dotenv.config() to read .env src/server.ts should start the app:

```
const PORT = process.env.PORT || 3000;
```

```
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

If these exist and run with:

```
ts-node src/server.ts
```

...then your backend is ready for Cypress tests.

1c: Check Database

1. Make sure your docker-compose.yml for Postgres exists and is simple:

```
version: "3.9"
```

```
services:
```

```
  db:
```

```
    image: postgres:15
```

```
  environment:
```

```
    POSTGRES_USER: test
```

```
    POSTGRES_PASSWORD: test
```

```
  POSTGRES_DB: testdb  ports:
```

```
    - "5432:5432"
```

2. Start Docker DB:

```
docker-compose up -d
```

3. Confirm the DB is running:

```
docker ps
```

You should see the postgres:15 container running.

4. Check .env is correct:

```
DATABASE_URL="postgresql://test:test@localhost:5432/testdb"
```

5. Run Prisma migrations if you haven't yet:

```
npx prisma generate
```

```
npx prisma migrate dev --name init --skip-seed
```

Now the DB tables exist and Prisma is ready.

1d: Why Step 1 is Important for E2E

- Cypress hits your **actual running server**, so the app and DB must be up.
- Without this, tests will fail because endpoints won't exist.

- Think of this step as “**make sure backend is alive**” before any test runs.

Step 2. Install Cypress

```
npm install -D cypress
```

Step 3. Open Cypress for the first time

```
npx cypress open
```

- Creates cypress/ folder and example tests
- Opens Cypress GUI

Configure Cypress

```
// cypress.config.ts
```

```
import { defineConfig } from "cypress";

export default defineConfig({
  e2e: {
    baseUrl: "http://localhost:3000",
    supportFile: "cypress/support/e2e.ts",

    setupNodeEvents(on, config) {
      // implement node event listeners here
    },
  },
});
```

- baseUrl → makes requests simpler (cy.request("/users") instead of full URL)

```
describe("Users API E2E", () => {  
  const user = { name: "Charlie", email: "charlie@test.com" };
```

```
  it("creates a new user", () => {  
    cy.request("POST", "/users", {  
      name: "Alice",  
      email: "alice@test.com",  
    }).then((response) => {  
      expect(response.status).to.eq(201);  
      expect(response.body).to.have.property("id");  
    });  
  });
```

```
  it("lists all users", () => {  
    cy.request("GET", "/users").then((response) => {  
      expect(response.status).to.eq(200);  
      expect(response.body.length).to.be.greaterThan(0);  
    });  
  });  
});
```

Step 4 : create above file in cypress/e2e/users.cy.ts

Step5: create run-all.js

```
const { spawn, execSync } = require("child_process");
```

```
const PROJECT = "test-env";
```

```
let serverProcess = null;
```

```
function run(cmd) {  
  console.log(`\n🔑 ${cmd}`);  
  execSync(cmd, { stdio: "inherit" });  
}
```

```
function wait(ms) {  
  return new Promise((res) => setTimeout(res, ms));  
}
```

```
// simple wait (no db push ❌)  
async function waitForDB() {  
  console.log("⌚ Waiting for DB...");  
  await wait(5000);  
}
```

```
async function startServer() {  
  console.log("\n🚀 Starting server...");  
  
  serverProcess = spawn("npx", ["tsx", "src/app.ts"], {  
    stdio: "inherit",  
    shell: true,  
  });  
}
```

```
// give server time to boot  
await wait(5000);  
}
```

```
async function stopServer() {  
  if (serverProcess) {  
    console.log("\n🛑 Stopping server...");  
    serverProcess.kill();  
  }  
}
```

```
(async () => {  
  try {  
    // 1. Start DB
```

```

run(`docker compose -p ${PROJECT} up -d`);

// 2. Wait for DB
await waitForDB();

// 3. Prisma setup (correct order)
run("npx prisma generate");
run("npx prisma migrate dev --name init");
run("npx prisma migrate reset --force");

// 4. Start backend
await startServer();

// 5. Run Cypress
run("npx cypress run");

} catch (err) {
  console.error("\n❌ Error:", err.message);
} finally {
  // 6. Cleanup
  await stopServer();

  console.log("\n🧹 Cleaning up Docker...");
  try {
    run(`docker compose -p ${PROJECT} down`);
  } catch (e) {
    console.error("Failed to stop Docker");
  }

  console.log("\n✅ Done");
}
})();

```

⚡ What run-all.js does

1 Start DB

docker compose up -d

🔗 Starts PostgreSQL in Docker

2 Wait

`wait(5000)`

👉 Gives DB time to start

3 Setup Prisma

`npx prisma generate`

`npx prisma migrate dev`

`npx prisma migrate reset --force`

👉 Creates client + creates tables + resets DB

4 Start backend

`npx tsx src/app.ts`

👉 Runs your Express server (port 3000)

5 Run tests

`npx cypress run`

👉 Runs API tests

6 Cleanup

`kill server`

`docker compose down`

👉 Stops server + removes DB container

🎯 One-line summary

👉 **Setup DB → prepare schema → start server → run tests → cleanup everything**

If you want even tighter:

👉 This is basically your **mini CI pipeline locally** 👍

[node run-all.js](#)

